Lecture 1: Getting started with supervised learning

Max G'Sell

Machine Learning I: 46-926

January 14, 2019

Goals of this course

- Become familiar with common machine learning tools and ideas, from both a theoretical foundation and an applied viewpoint
- Be able to recognize a problem and develop a useful approach to modeling it, and then actually carry it out
- Become comfortable with the fundamentals of machine learning, so that more complicated approaches can be understood and incorporated in the future

This course: practical aspects

- 6 homeworks, due on canvas at the beginning of classes 2-7
  - The homeworks will tend to require a significant effort...
- Final exam
- Notes will be posted before class so you can write on them. Scanned/tablet versions of the notes will be posted after class (by next day).

Homeworks

Homework problems take three main forms:

- Theoretical problems to understand how things work and to become familiar with important concepts and tools.
- Simulations to see how methods perform – and more often, how they break down.
- Data examples to see how to run methods and to build an intuition about what you would see on real data. Also helps to understand problems that arise in practical settings and how to fix them.

Why Theory?

- Real data is messy and always presents new complications
- Understanding why and how things work is a necessary precursor to figuring out what to do.
  - When does a method apply.
  - What might make it fail?
- Provides building blocks to understanding or even designing more complicated approaches.

Collaboration

Talk about the homeworks, help each other learn. However, you need to:

- Write your own code. Making debugging suggestions is ok. Writing one version of the code together is not.
- Do your own writeup of the math. You can talk about and sketch ideas, but you shouldn't be looking at someone else's work when you are writing your own.

You should be able to explain anything that you submit...

Discussion

There is a Piazza discussion board through canvas. Please use this instead of email for any questions about the material or homework.

This way there is a fair distribution of information, and you can also help each other learn.

I also tend to be faster at Piazza than at email. However, I make no promises with either very close to homework deadlines.

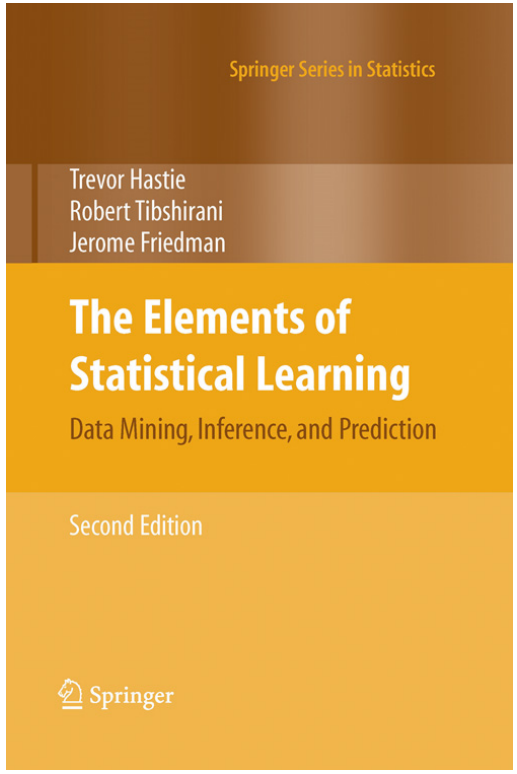
Books

Two useful references for this course are

- *Elements of Statistical Learning* by Trevor Hastie, Robert Tibshirani, and Jerome Friedman.

- *Introduction to Statistical Learning* by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. It is easier to read and has more code examples, but covers less material and is written at a lower level.

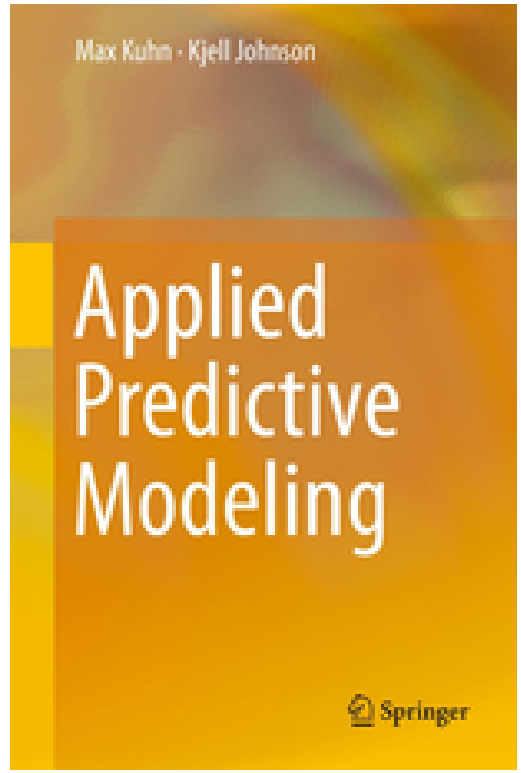
  Both textbooks are available (legally) for free online from the authors. Both are also available from SpringerLink if you are on the CMU network.



Books

The book *Applied Predictive Modeling* by Max Kuhn and Kjell Johnson is also a useful reference for some of the more practical aspects of applying machine learning.

This book is available free online from SpringerLink if you are on the CMU network.



Subject matter

- While this is an machine learning class with a focus on finance, our primary goal will be the machine learning. If there is a chance to illustrate an interesting concept with a non-finance data set, I will do so.
  - Many methods were originally developed for other uses, which can lend a better intuition.
  - Other fields often have higher signal-to-noise, making for a cleaner illustration of new ideas.
  - Interesting new applications in finance take time to spread.

- The course is planned to be done primarily in python. However, for some methods R is better suited, so we will fall back on that from time to time as an alternative or just for comparison.

Announcements

- There will be an announcement slide at the beginning of each lecture. You are responsible for being aware of this content, even if you miss class.
- Homework 1 is out tomorrow and due on Monday.

Today: Intro to ML and Supervised Learning

- The basic terminology and intuitions behind Machine Learning
- Returning to linear regression
- Extending linear regression

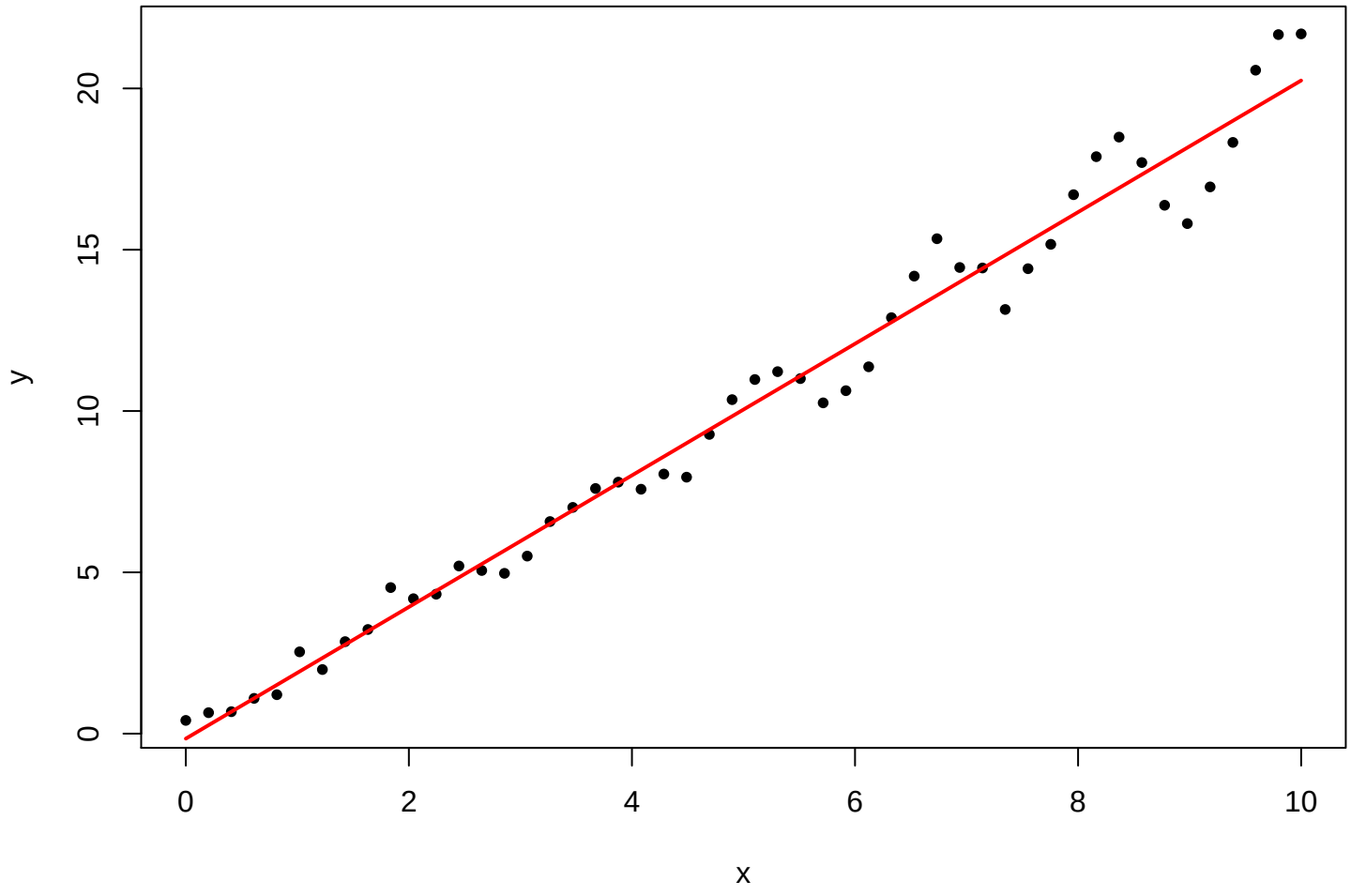
Classification of data mining problems

Machine learning problems are often divided as follows

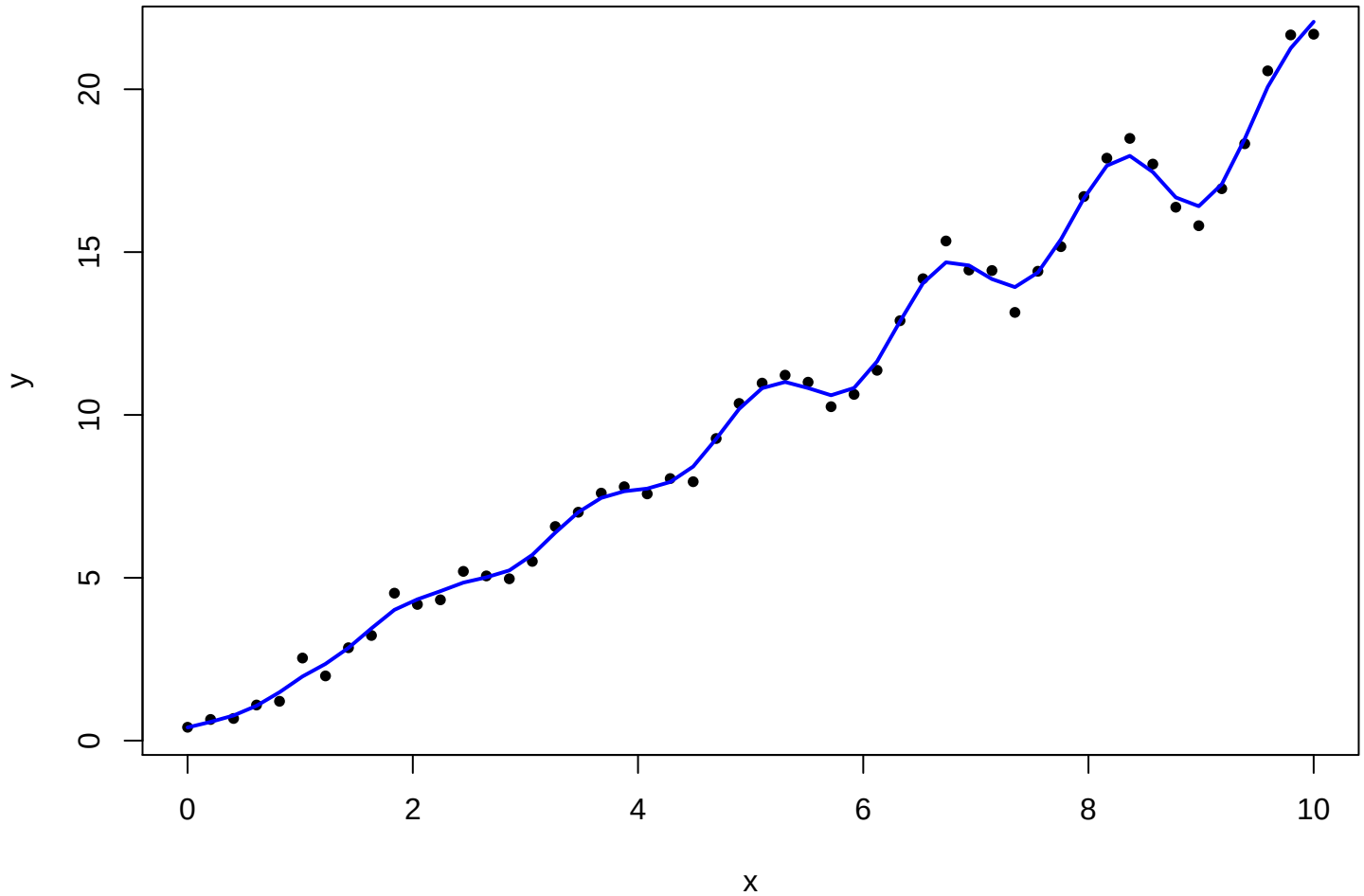
- Supervised learning: Making predictions
  i.e., given measurements $(X_1, Y_1), \ldots (X_n, Y_n)$, learn a model to predict Y_i from X_i
  - Prediction: Y_i is a continuous value
  - Classification: Y_i is a discrete value

- Unsupervised learning: discovering structure
  E.g., given measurements $X_1, \ldots X_n$, learn some underlying structure based on similarity

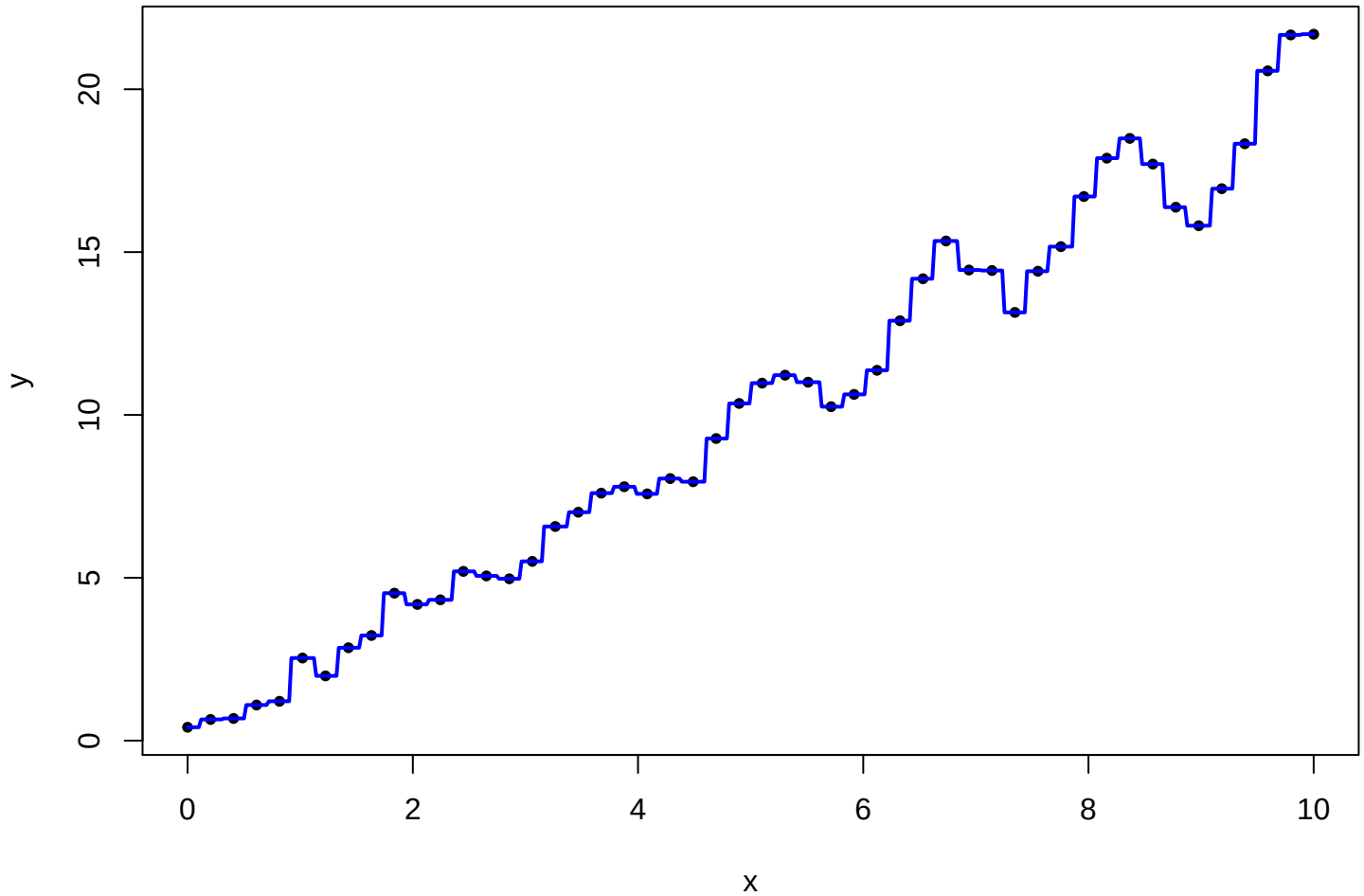
Supervised learning: Linear regression



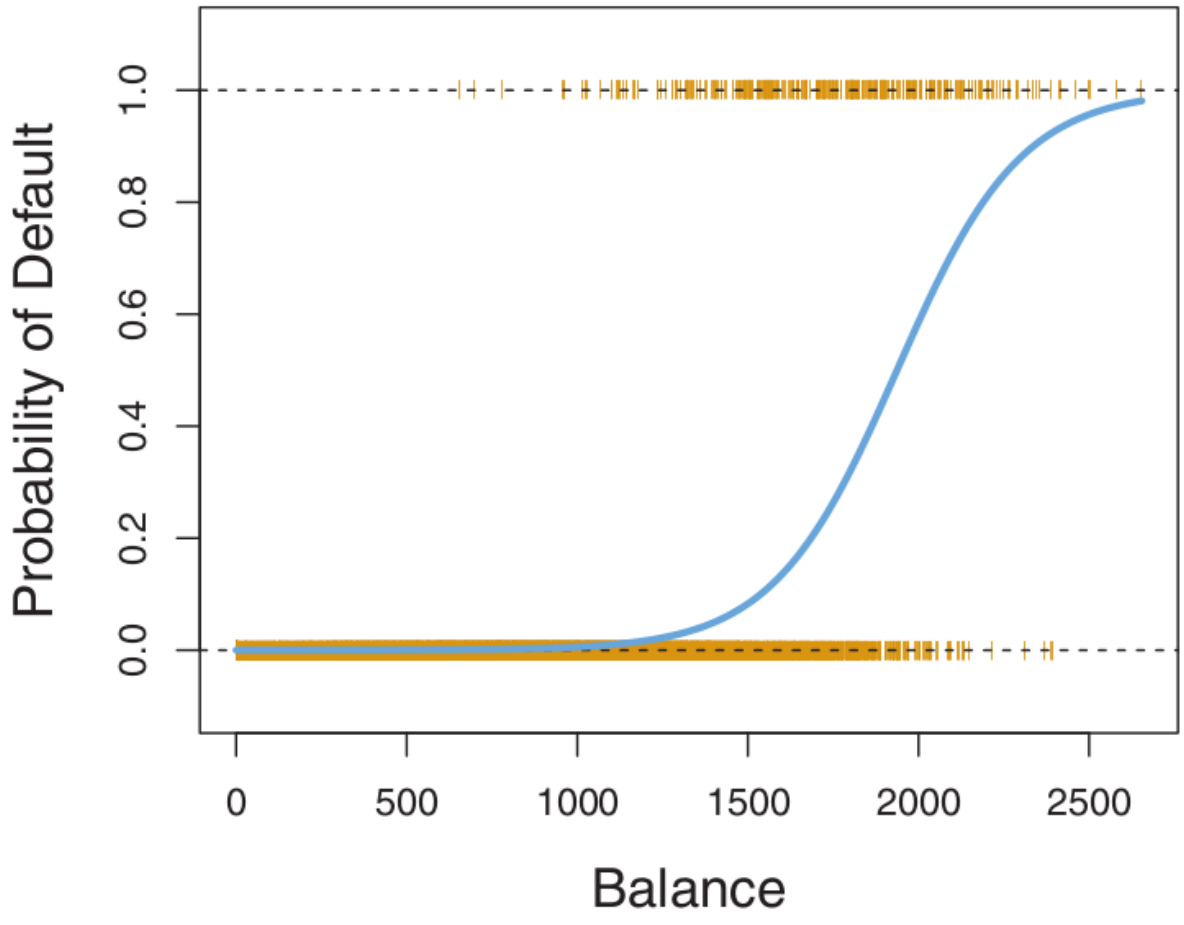
Supervised learning: Splines



Supervised learning: k-nearest neighbors

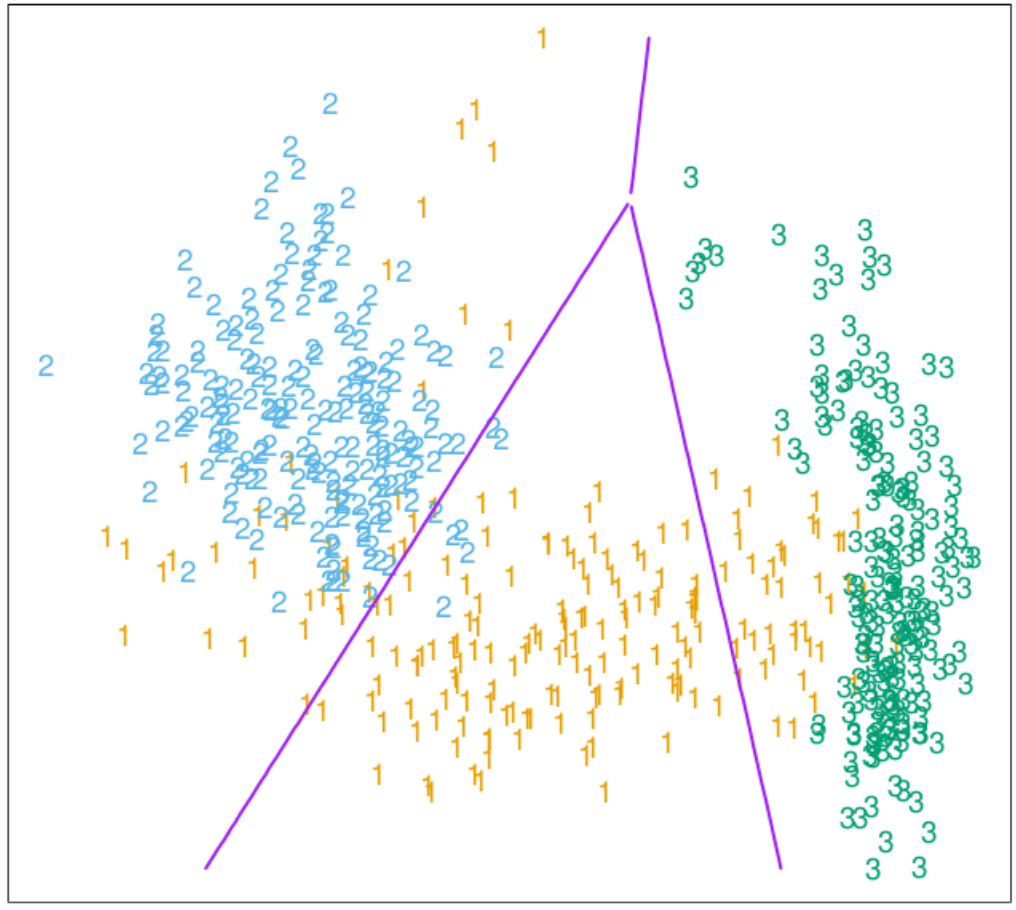


Supervised learning: Logistic regression



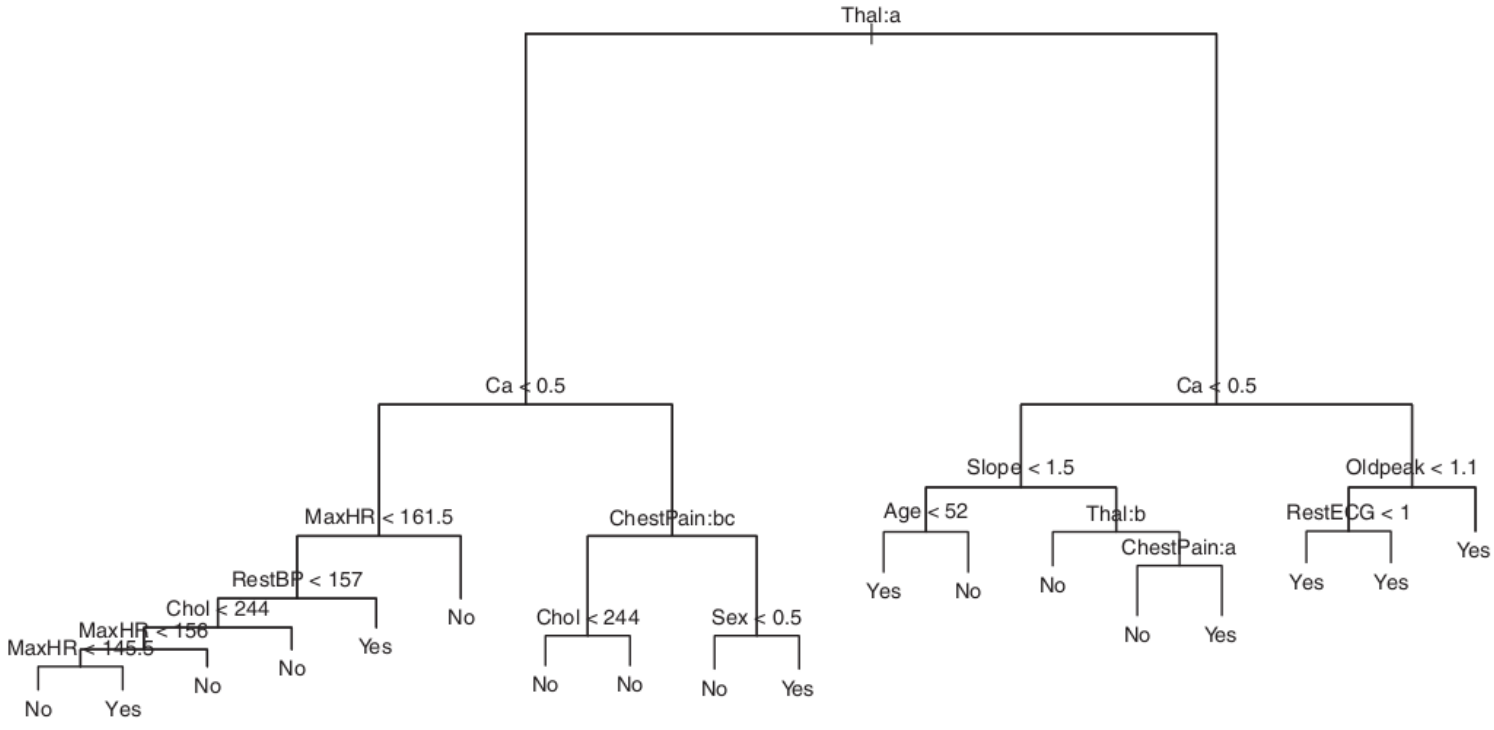
(ISL pg. 131)

Supervised learning: Linear discriminant analysis



(ESL pg. 103)

Supervised learning: Decision trees, forests, boosting



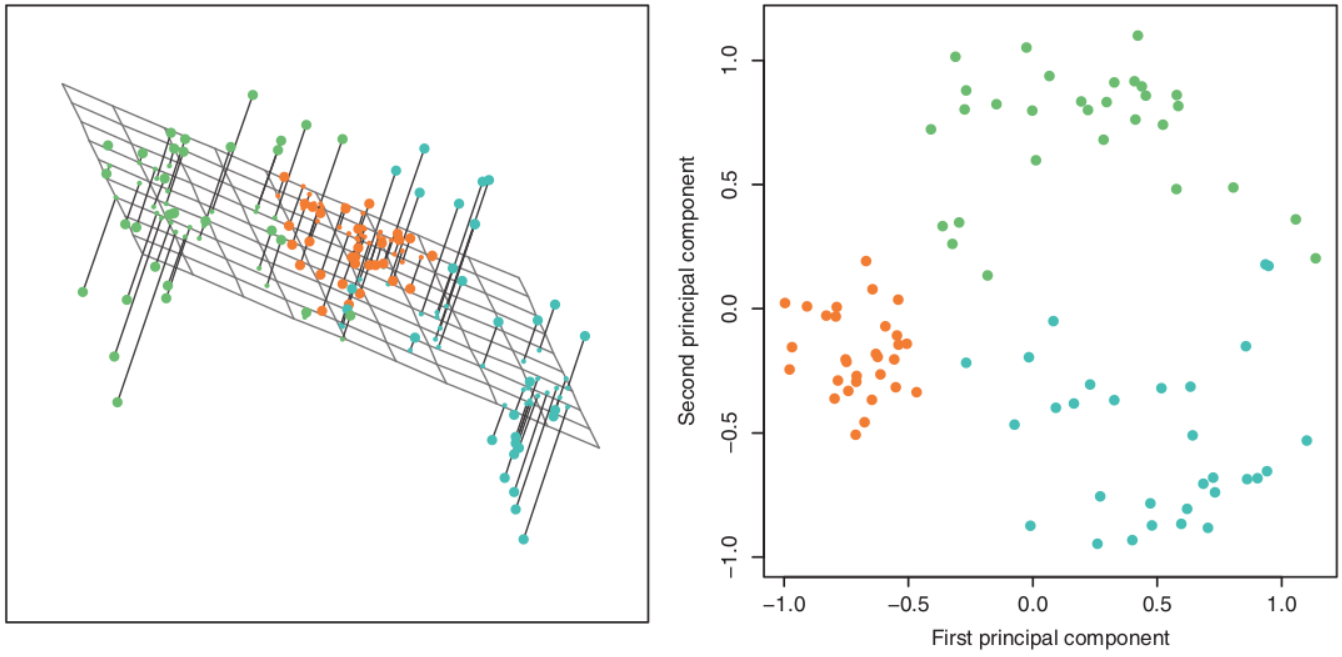
(ISL pg. 313)

Unsupervised learning

Unsupervised learning sounds a little more vague. What does it mean to "discover underlying structure"?

- Dimension reduction Discover lower-dimensional structure in high dimensional data
- Clustering Detect groups of similar units or variables

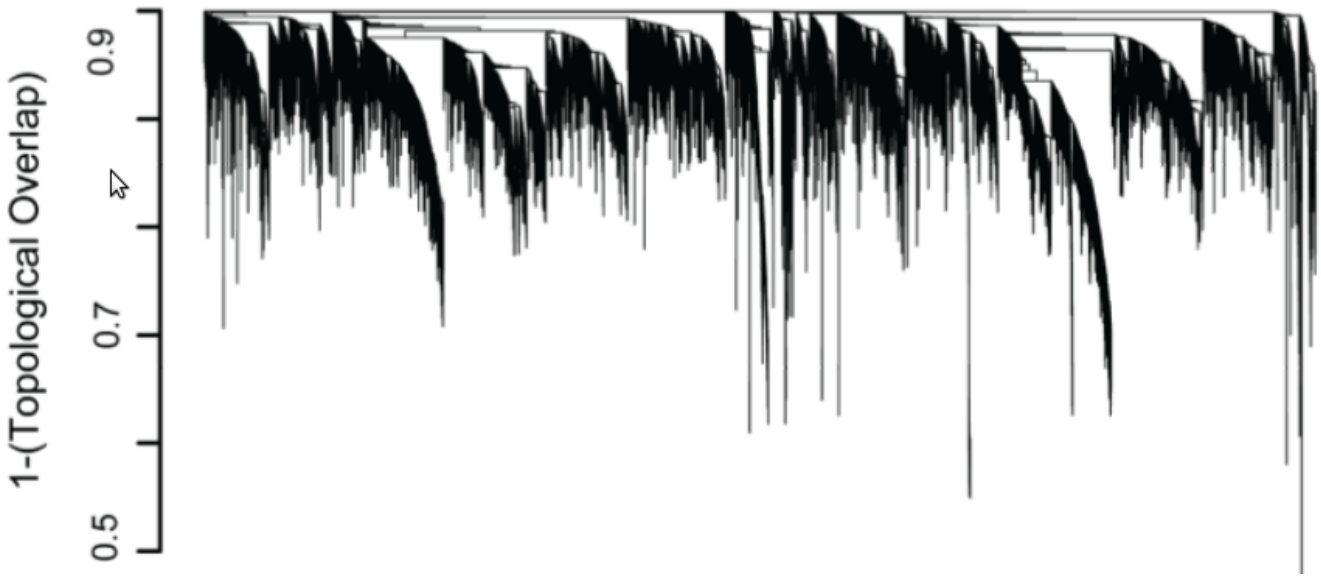
Unsupervised learning: PCA



(ISL pg. 131)

Discover lower-dimensional structure in high dimensional data

Unsupervised learning: Clustering



(Parikshak et. al., 2013)

Detect groups of similar units or variables

Supervised/unsupervised learning

These divisions are not absolute. There is plenty of crossover and ambiguity.

- Is search a supervised or unsupervised problem?
- If I detect low dimensional structure in the covariates, shouldn't I be able to make better predictions?
- Reinforcement learning learns a policy in an online fashion, sampling input/output pairs to both:
  - Learn about the system
  - Maximize payoff, including these samples

Nevertheless, the terms provide a useful approach to thinking about problems and putting them in a reasonable framework.

Supervised learning setup

In the basic prediction/classification ML setup, we observe independent draws (x, y) from a fixed distribution $\mathcal{P}$, with $x \in \mathbb{R}^p$.

- In *prediction*, $y \in \mathbb{R}$
- In *classification*, $y \in \{1, \ldots, K\}$, or some other discrete set.

In the simplest setting, we see n examples, $(x_1, y_1), \ldots, (x_n, y_n)$. From these, we want to build a function $\hat{\mu}(x)$ that takes vectors $X \in \mathbb{R}^p$ to guesses at Y.

- X as something we can get, either because it happened in the past or because it's cheap to measure.
- Y is something we want but is hard to get, either because it will happen in the future or is expensive to measure.

In later lectures, we will see interesting modifications of this setup, and the resulting complications.

What makes a good $\hat{\mu}$?

Our guesses $\hat{\mu}(x)$ will never be perfect. The *loss function*, $\mathcal{L}(Y, \hat{\mu}(X))$, represents the cost of guessing $\hat{\mu}(X)$ when the true value is Y (for a new (X, Y) from $\mathcal{P}$).

For convenience, we often use *squared error loss* for prediction,

$$\mathcal{L}(Y, \hat{\mu}(X)) =$$

and 0 - 1 loss for classification,

$$\mathcal{L}(Y, \hat{\mu}(X)) =$$

Note that these are rarely your actual losses. Nevertheless, they are useful for thinking about and developing methods.

Example: Realistic Loss

Throughout this class (and machine learning as a whole), we make many idealizations, including our loss functions.

This is fine, as long as you understand this limitation and keep it in mind. You should always think about what your actual loss, and at least stop at the end to consider the consequence of ignoring it.

Example: In Statistical Arbitrage, you will consider classifying stocks into "winners" and "losers". You will then build portfolios where you are long in the winners and short in the losers.

How should you really think about loss in this example? Is zero-one loss correct? Is it useful?

Expected Loss

Of course, it's going to be hard to assess $\mathcal{L}(Y, \hat{\mu}(X))$ directly, since it looks at a new (X, Y) pair and we don't get to see Y.

Instead, we need to focus on a summary measure that's easy to estimate. We tend to use *expected loss*, $\mathbb{E}\mathcal{L}(Y, \hat{\mu}(X))$.

What is random here?

- Imagine sampling a data set, X, Y.
- Fitting a $\hat{\mu}$ to that data.
- Drawing a point X, Y to evaluate the function at.

Expected Loss

This leads to the most common metrics one tries to optimize:
Expected prediction error (prediction):

$$\mathbb{E}\mathcal{L}(Y, \hat{\mu}(X)) =$$

Expected misclassification error (classification):

$$\mathbb{E}\mathcal{L}(Y, \hat{\mu}(X)) =$$

Even though these may not completely reflect our true costs, they give us a very useful way to think about machine learning. In particular, they lead to two simple, important insights.

Regression function

Suppose we had our dream, and we actually knew $\mathcal{P}$. What would be the best function μ we could possibly use? Let's minimize the expected prediction error. Here X, Y are random, but not μ!

$$\mathbb{E}\mathcal{L}(Y, \hat{\mu}(X)) =$$

$$=$$

$$=$$

$$=$$

The first term doesn't depend on μ. Minimizing the other two with respect to $\mu(X)$ gives

$$\mu(X) =$$

This is our best possible choice of $\mu(X)$! However, we don't know this quantity without knowing $\mathcal{P}$, so we instead try to estimate it from our data.

Bias-variance decomposition

To make notation simpler, write $\mu(x) = E(Y|X = x)$, our regression function. We are now estimating μ with $\hat{\mu}$ on a separate data set, so that is random too.

Then

$$\mathbb{E}\mathcal{L}(Y, \hat{\mu}(X)) =$$

Define $\sigma_x^2 =$

Bias-variance decomposition

$$\mathbb{E}\mathcal{L}(Y, \hat{\mu}(X)|X = x) =$$

$$=$$

$$=$$

$$=$$

$$=$$

$$\text{bias}\left(\hat{\mu}(x)\right) =$$

$$\text{var}\left(\hat{\mu}(x)\right) =$$

Bias-variance decomposition

$$\mathbb{E}\left((Y-\hat{\mu}(X))^2\,|X=x\right)=\sigma_x^2+\text{bias}\left(\hat{\mu}(x)\right)^2+\text{var}\left(\hat{\mu}(x)\right)$$

This gives you a way of thinking about where errors in predictions come from.

If your estimator is too inflexible to approximate the true mean function, it is too

If your estimator
adapts too much to individual training samples and over fits, it is too

We will see many examples.

Example: Linear regression

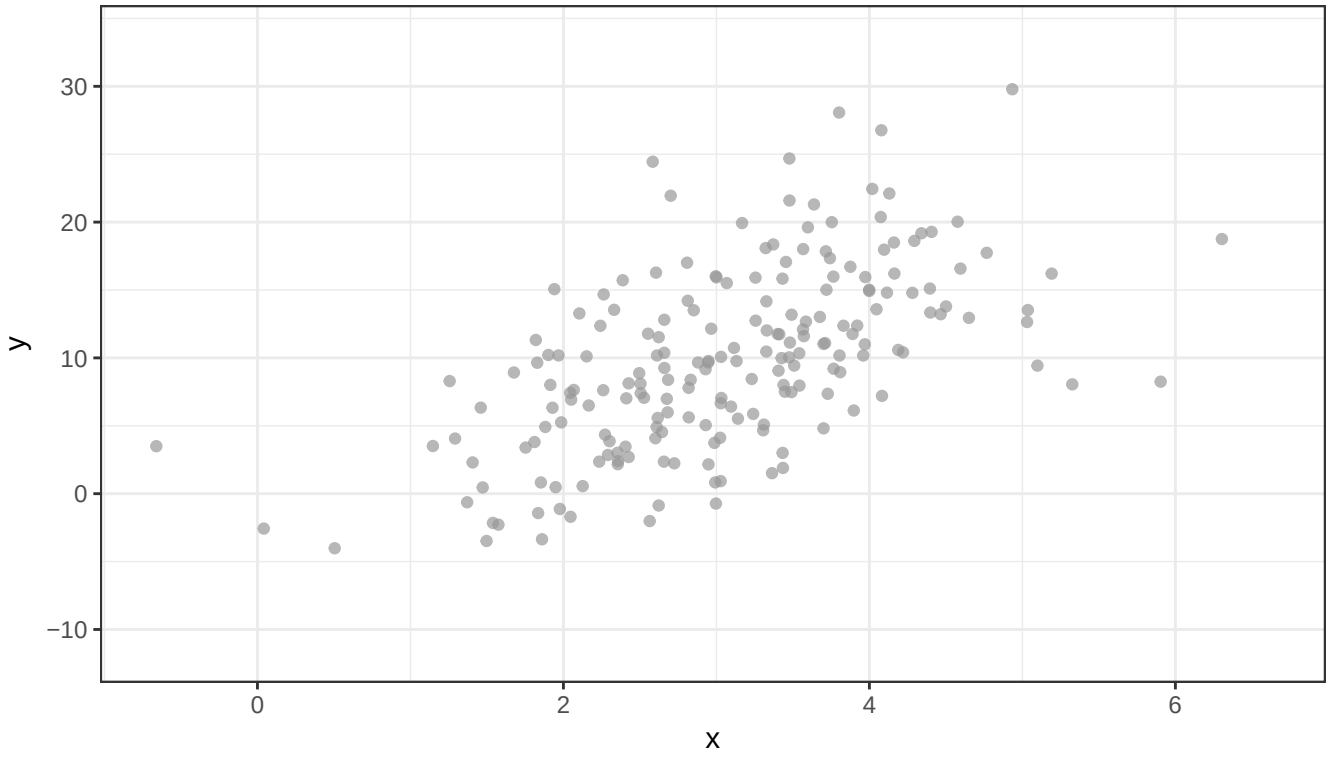
For machine learning purposes, we think of regression as a very simple way to approximate $\mathbb{E}(Y|X)$. While $\mathbb{E}(Y|X)$ could be any function, we decide to approximate it as a linear function of the predictors.

$$\begin{aligned}\hat{\mu}(x) = &= \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p \\ &= x^T \hat{\beta}\end{aligned}$$

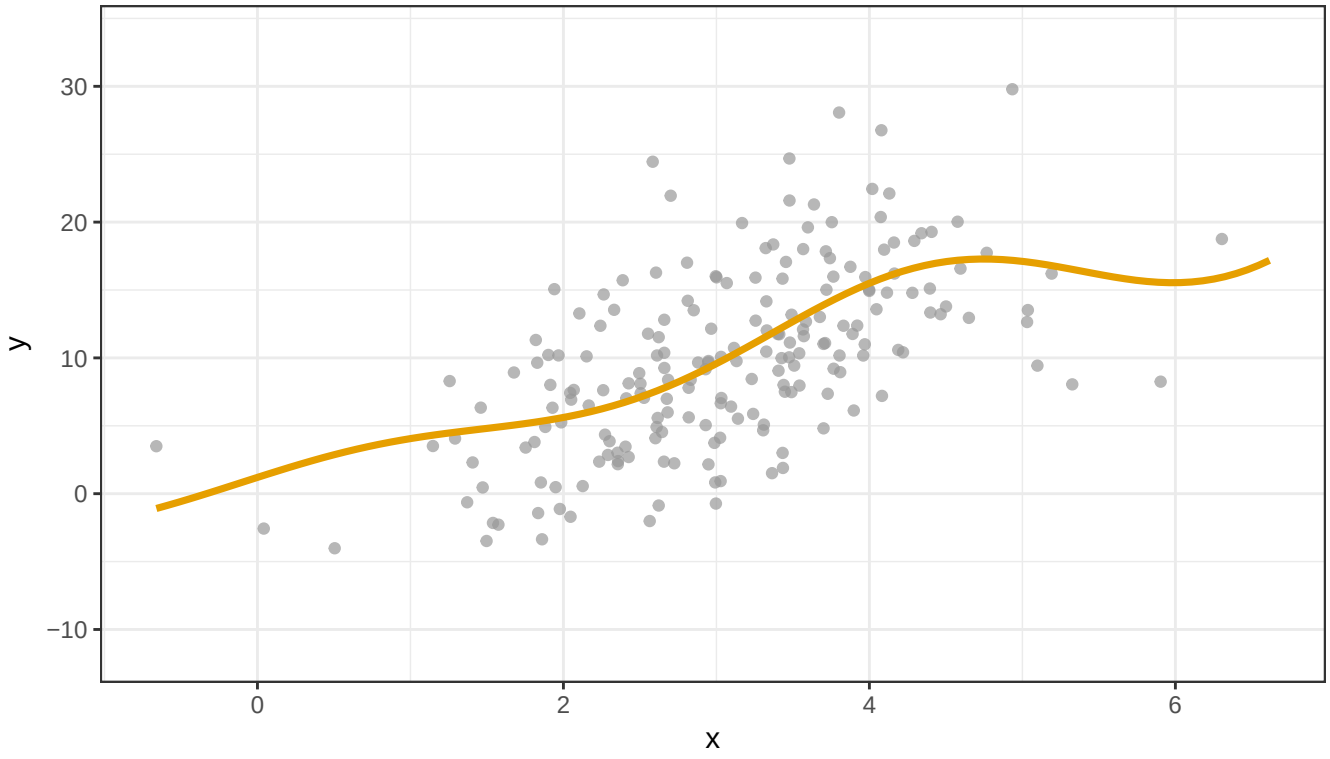
This simplifies things tremendously, but also introduces many problems. We usually estimate $\hat{\beta}$ using least squares, as you learned.

This is different from the common statistical view of regression. *We don't need to believe that the linear model assumptions are at all true!* We just hope it makes good guesses.

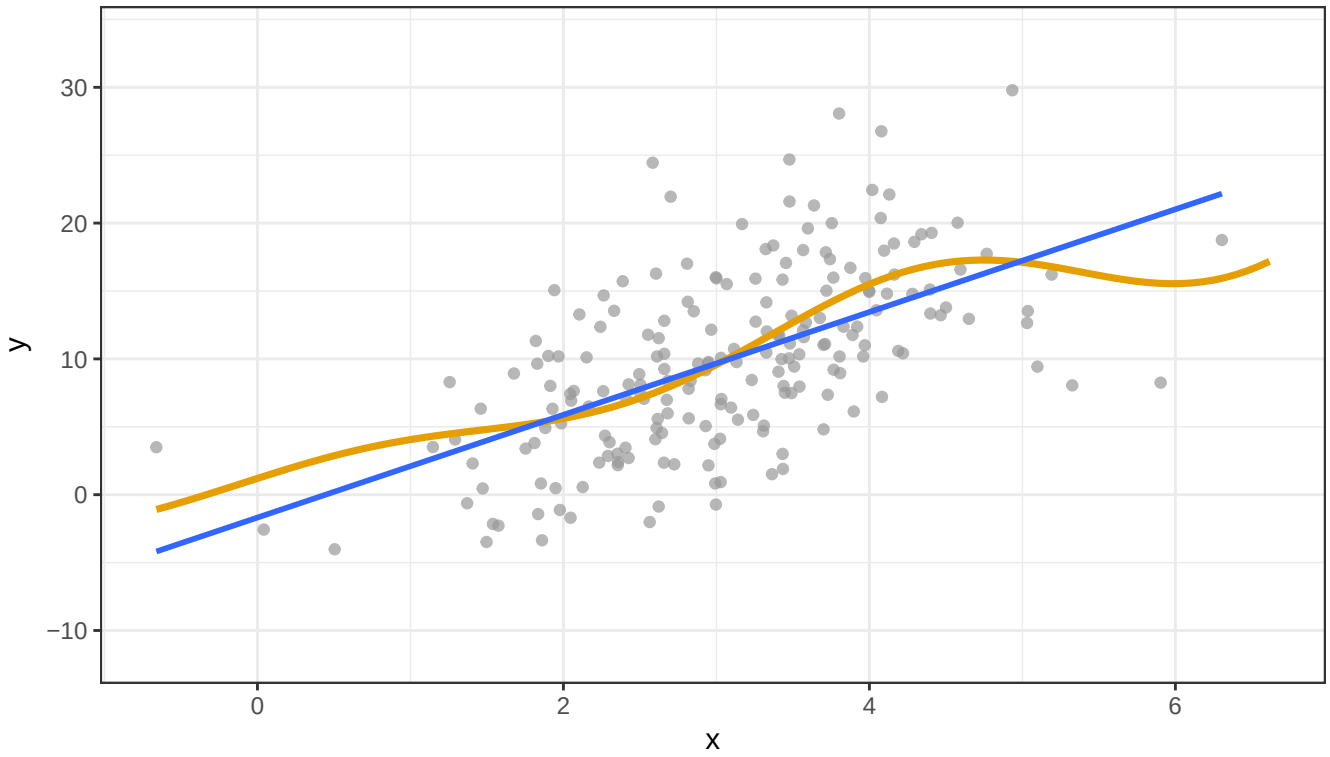
Regression function demo



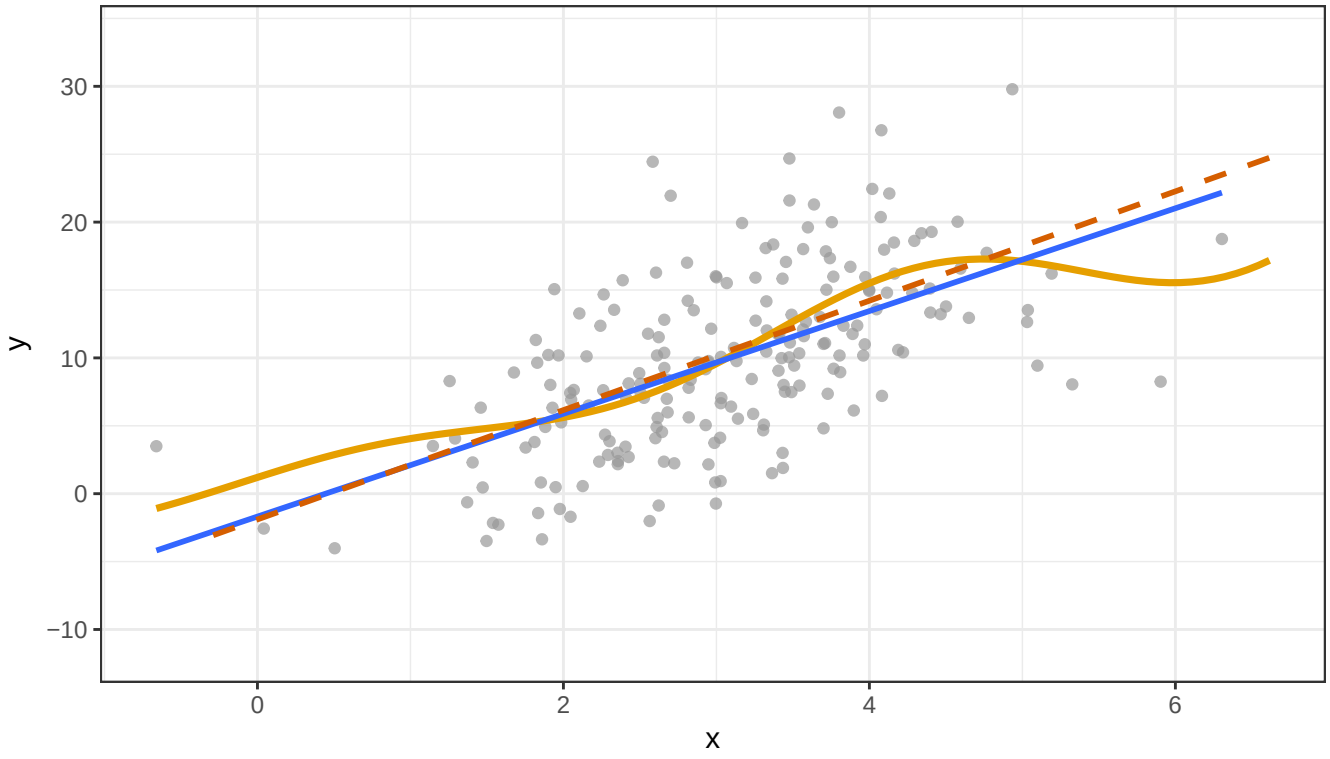
Regression function demo



Regression function demo



Regression function demo



Linear regression

$$\begin{aligned}\hat{\mu}(x) = &= \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p \\ &= x^T \hat{\beta}\end{aligned}$$

If the true $\mathbb{E}(Y|X)$ is very far from linear, this model will be highly *biased*, since even $\mathbb{E}\hat{\mu}(X)$ will never be able to match $\mathbb{E}(Y|X)$.

Even with infinite data, the best linear model would still be terrible. You learned (and we will discuss) much more flexible models to avoid this problem.

We will have other problems even when the linear model is correct!

Review: regression with multiple predictors

Last semester, you talked about regression with one covariate:

$$Y = \beta_0 + \beta_1 X + \varepsilon.$$

You didn't get a chance to talk about multiple linear regression:

$$Y = \beta_0 + \beta_1 X_1 + \ldots \beta_p X_p + \varepsilon,$$

so we will take a bit of time to talk about the *statistical* properties as we go.

Review: regression with multiple predictors

Suppose that we observe n copies of $(X_1, \ldots, X_p, Y)$. We can fit the model

$$Y = \beta_0 + \beta_1 X_1 + \ldots \beta_p X_p + \varepsilon,$$

just as we did with one variable, by minimizing the RSS:

$$\begin{aligned} RSS(\beta) &= \sum_{i=1}^{p} (y_i - \hat{y}_i)^2 \\ &= \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij} \beta_j \right)^2 \end{aligned}$$

You could just minimize this by brute force calculus, but we're going to take a more pleasant and insightful approach.

(Note: I'm not going to bother with the intercept to keep notation pleasant, you can just pretend that $X_1 = 1$.)

Review: regression with multiple predictors

Suppose that we observe n copies of $(X_1, \ldots, X_p, Y)$. We can stack all the X values into a matrix $\mathbf{X}$,

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}$$

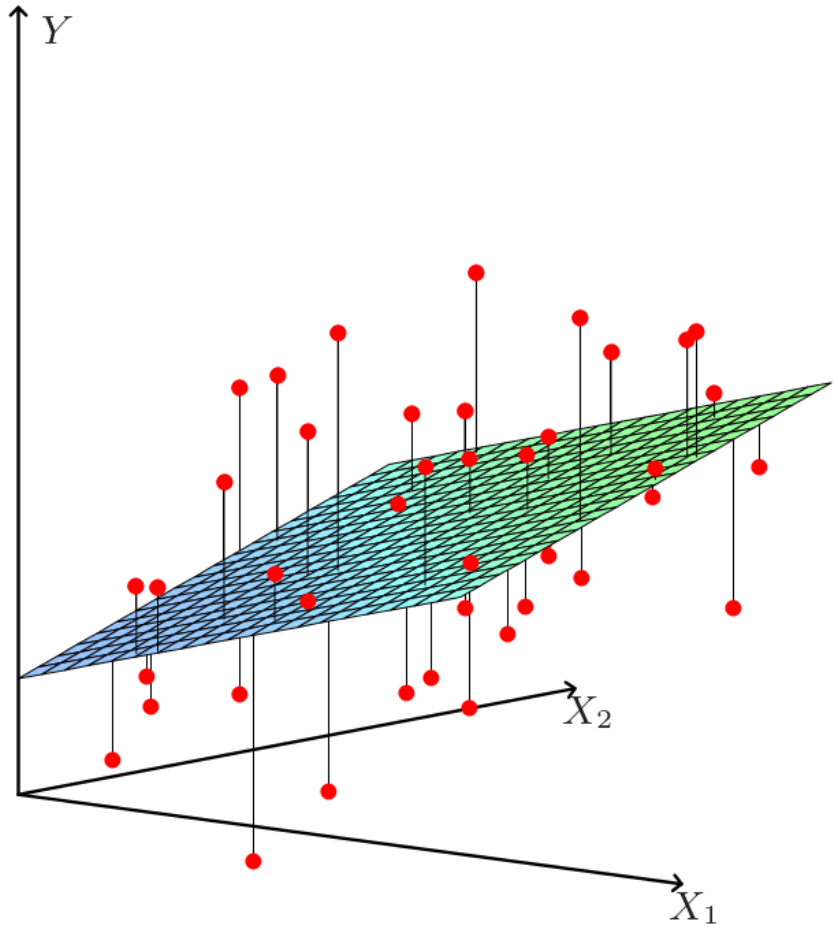
Note that the rows $x_i \in \mathbb{R}^p$, $x_i = (x_{i1}, \ldots, x_{ip})$, are the individual observations, and the columns $\mathbf{x}_j \in \mathbb{R}^n$, $\mathbf{x}_j = (x_{1j}, \ldots, x_{nj})^T$, are all the observations of a particular variable.

We can then write the corresponding linear model for all the Y very simply as $Y = X\beta$, or expanded,

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} \begin{pmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{pmatrix}$$

Why do we care? The linear algebra reformulation of the problem gives us geometric intuition into regression, which will carry over into more complicated ML procedures.
It will also make our derivations easier here.

Two Pictures: p-space



$$\begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}, \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

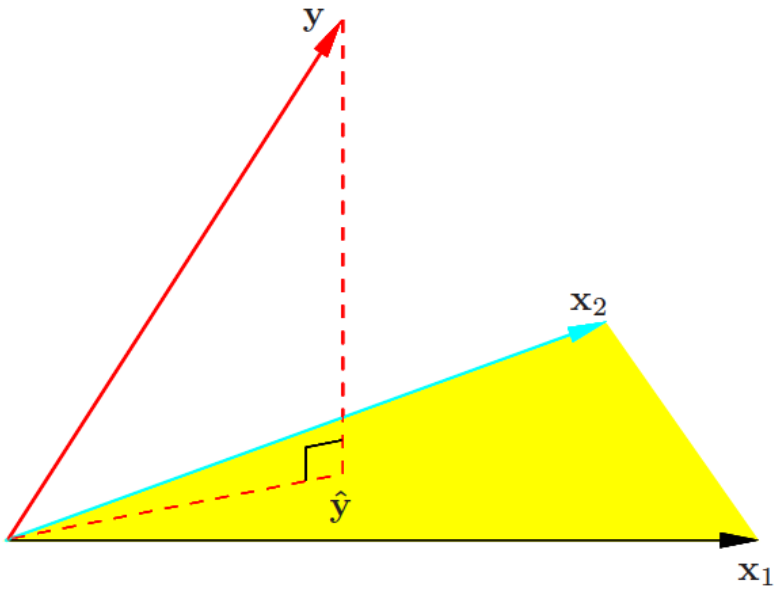
Views the observations $(x_{i1}, \ldots, x_{ip}, y_i)$ as points in $\mathbb{R}^p$

$$y_i = \sum_{i=1}^{p} x_{ij}\beta_j = x_i^T \beta$$

Easy to see:

- What a new observation does
- How well individual y_i are approximated

Two Pictures: n-space



$$\begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}, \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

Views the columns $(x_{1j}, \ldots, x_{nj})$ as vectors in $\mathbb{R}^n$, along with $(y_1, \ldots, y_n)$ and $(\varepsilon_1, \ldots, \varepsilon_n)$

$$Y = \sum_{i=1}^{n} X_j \beta_j$$

Easy to see:

- How to fit our model
- What happens when new variables are added?
- How the error ε interacts with dimension?

Minimizing the RSS now means minimizing

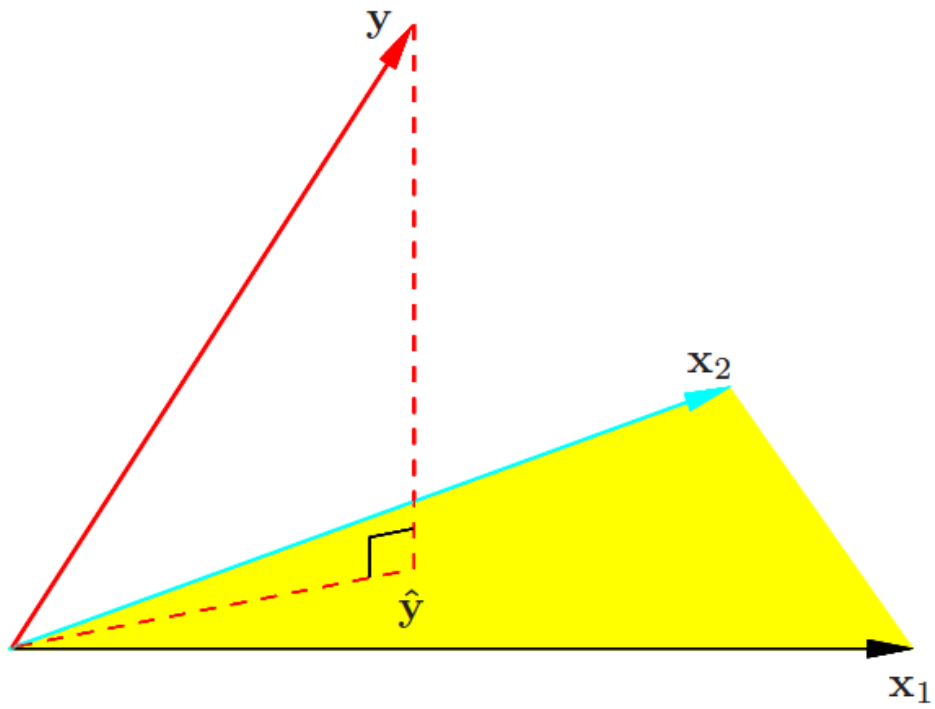
$$RSS(\beta) = \sum_{i=1}^{n} \left(y_i - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2$$

$$=$$

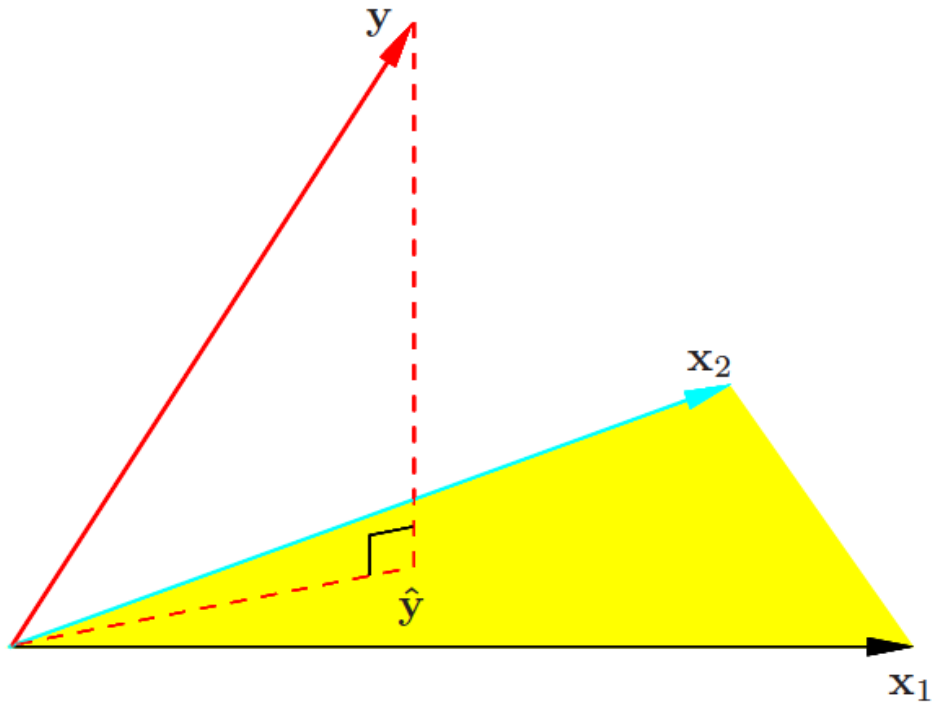
$$=$$

Let's get an intuition for how $\hat{Y} = X\beta$ behaves.

We can write $\hat{Y} = X\hat{\beta} = \sum_{j=1}^{p} \hat{\beta}_j X_j$. The last term is clearly a linear combination of the columns of X, so we know that $\hat{Y}$ lies in the column span of X (a linear subspace).



(ESL pg. 46)



$\hat{Y}$ minimizes $\|Y - \hat{Y}\|_2^2$, the euclidian distance from Y to $\hat{Y}$.

Since $\hat{Y}$ is confined to the column space, this will be minimized at the closest point to Y in the subspace, meaning that $\hat{Y}$ is the orthogonal projection of Y onto the column span of X.
So $\hat{Y} = P_X Y$.

If I claim to you that

$$P_X =$$

is the orthogonal projection operator onto the column span of X, then

$$\hat{Y} =$$

and therefore the minimizing parameters are given by

$$\hat{\beta} =$$

We usually write $H = X(X^T X)^{-1} X^T$ and call this the hat matrix. Note that $\hat{Y} = HY$ is a linear transformation.

Let's prove it carefully. First, show that H is an orthogonal projection. We need to show that:

- H is symmetric ($H = H^T$):

- H is idempotent ($H^2 = H$):

Finally, we need to show that it leaves anything in $\mathrm{col}(X)$ alone, and kills anything orthogonal to $\mathrm{col}(X)$:

- Suppose $y \in \mathrm{col}(X)$, so that $y = Xa$ for some $a \in \mathbb{R}^p$.

- Suppose $y \perp \mathrm{col}(X)$, so that $y \perp X_i$ for all i.

Therefore H is the orthogonal projection onto $\mathrm{col}(X)$.

This implies a bunch of facts quite easily.

- How are $\hat{Y}$ and $Y - \hat{Y}$ related?
- How do the residuals $Y - \hat{Y}$ relate to X? What happens if we regress them on X?
- What happens if we regress $\hat{Y}$ again on X?

Regression: Traditional statistics vs. ML

Suppose that you observe your data X, Y, and fit a vector of coefficients $\hat{\beta}$. In traditional statistics you would do things like:

- Produce an interval $[L, U]$ that is likely to contain the next Y_i (prediction intervals)

- Determine how large β really is in the "true" model that generated the data. (Confidence intervals, hypothesis testing)

Regression: Traditional statistics vs. ML

To answer these questions, you need:

1. A model you believe for generating data, say:

$$Y_i = x_i^T \beta + \varepsilon_i$$

where ε is mean zero and variance σ^2, and independent across observations. Or maybe $\varepsilon_i \sim N(0, \sigma^2)$
This leads us to worry about diagnostics like residual plots, which might show that our linear model is wrong!

2. Distributions of $\hat{\beta}$ for given true β. Example:

$$\hat{\beta} \sim N(\beta, \sigma^2 (X^T X)^{-1})$$

A variety of results of this sort exist for different model assumptions.

Regression: Traditional statistics vs. ML

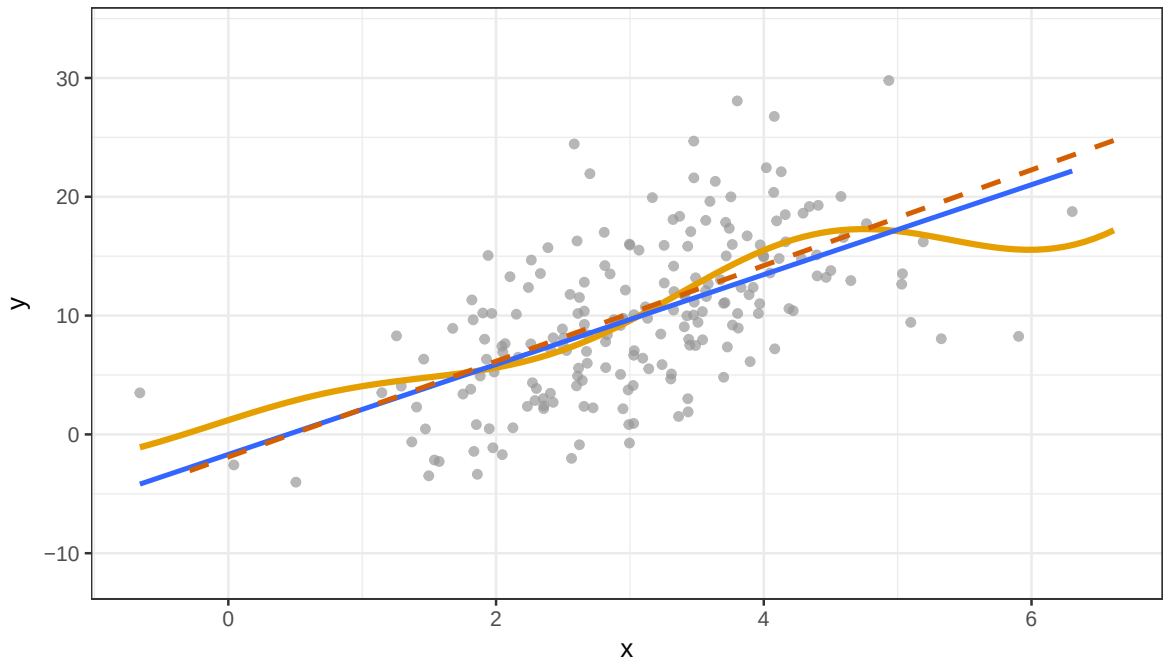
To get satisfying answers, you also worry about things like colinearity, meaning high correlations between predictors.

This is not really a problem for the math, but it tends to lead to difficulties in interpretation and in inference.

A prediction perspective

We care about all of the things above because we are trying to say something about the underlying "true" model.

From a prediction perspective, we might not really care about that true model!



Goals in modeling

In statistics and machine learning, we might fit a function $\hat{f}(x)$ to approximate a target $f(x)$. We could have several goals:

- Inference: Ask questions about $f(x)$.
- Interpretability: Ask questions about $\hat{f}(x)$
- Prediction: Guess Y using $\hat{f}(X)$.

Much of statistical inference focused on the first one. We will primarily care about the last two.

Goals in modeling

You have heard many horror stories about *colinearity* in regression. Suppose you fit a regression of Y on $X_1, X_2, \ldots$, but X_1 and X_2 are almost the same.

What happens to inference for linear regression?

What happens to predictions based on linear regression?

Linear regression as a predictor

Ok, so suppose that we use linear regression just to make predictions. What will happen?

You can obtain estimates $\hat{Y} = X\hat{\beta}$, and they might work well sometimes, even when the linear model is not true!

In particular, we know that least squares has nice properties when the linear model is true – e.g., it is unbiased and has minimum variance among unbiased, linear estimates (Gauss-Markov Theorem).

However...

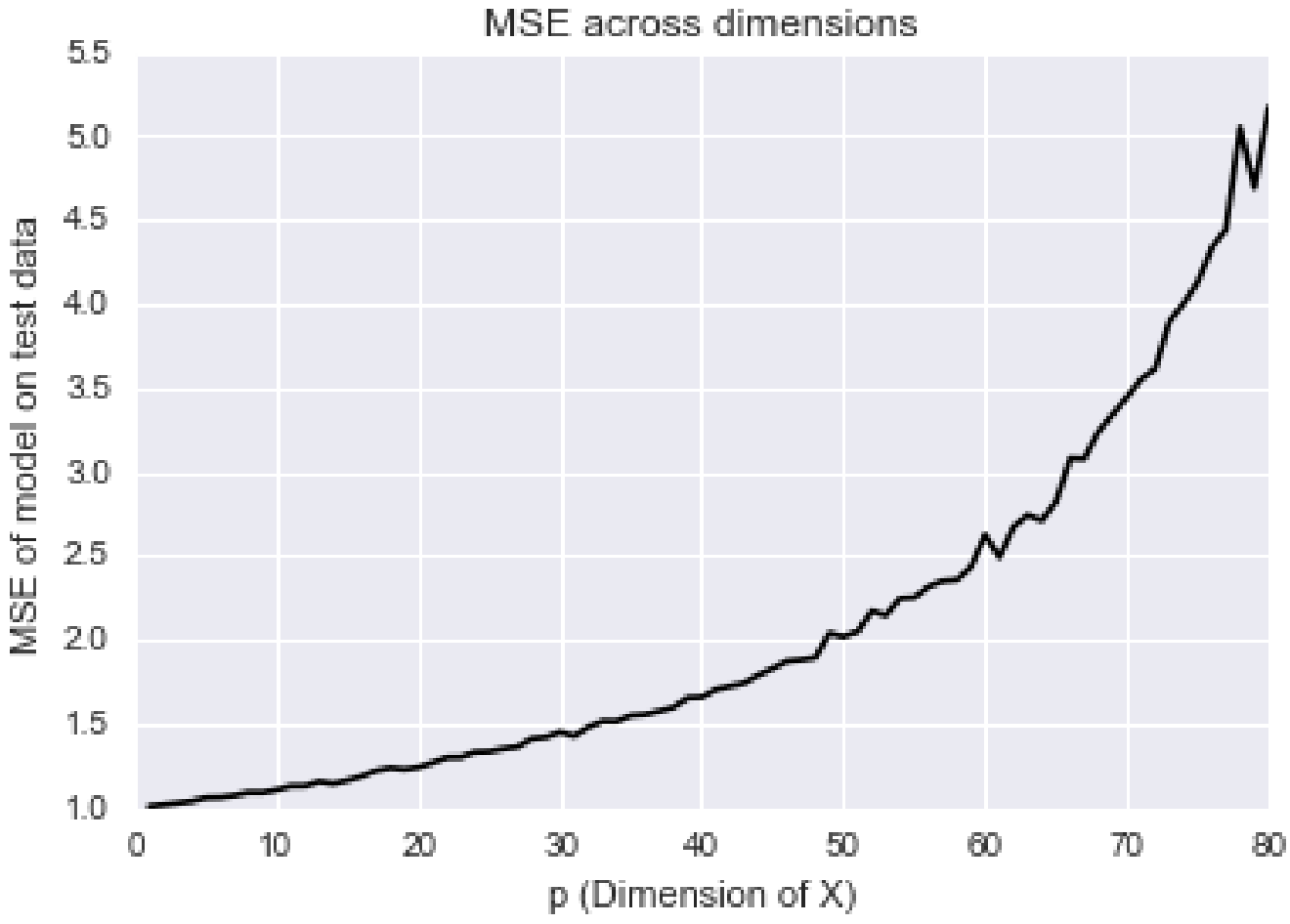
Linear regression in high dimensions

What happens when we have a bunch of variables? (Big data!)

We know that a linear model may perform terribly if the true function is far from linear.

Suppose that the linear model is even correct, so that $\mathbb{E}(Y|X)$ is exactly linear. What happens when we try to estimate it?

Linear regression in high dimensions



This is a simulation from a true linear model with independent Gaussian errors – the dream!

Linear regression in high dimensions

Suppose that the linear model is even correct, so that $\mathbb{E}(Y|X)$ is exactly linear. What happens when we try to estimate it?

As dimension increases, the prediction error increases (super-)linearly! When dimension is bigger than the number of observations,

What is going on, and how can we fix it?

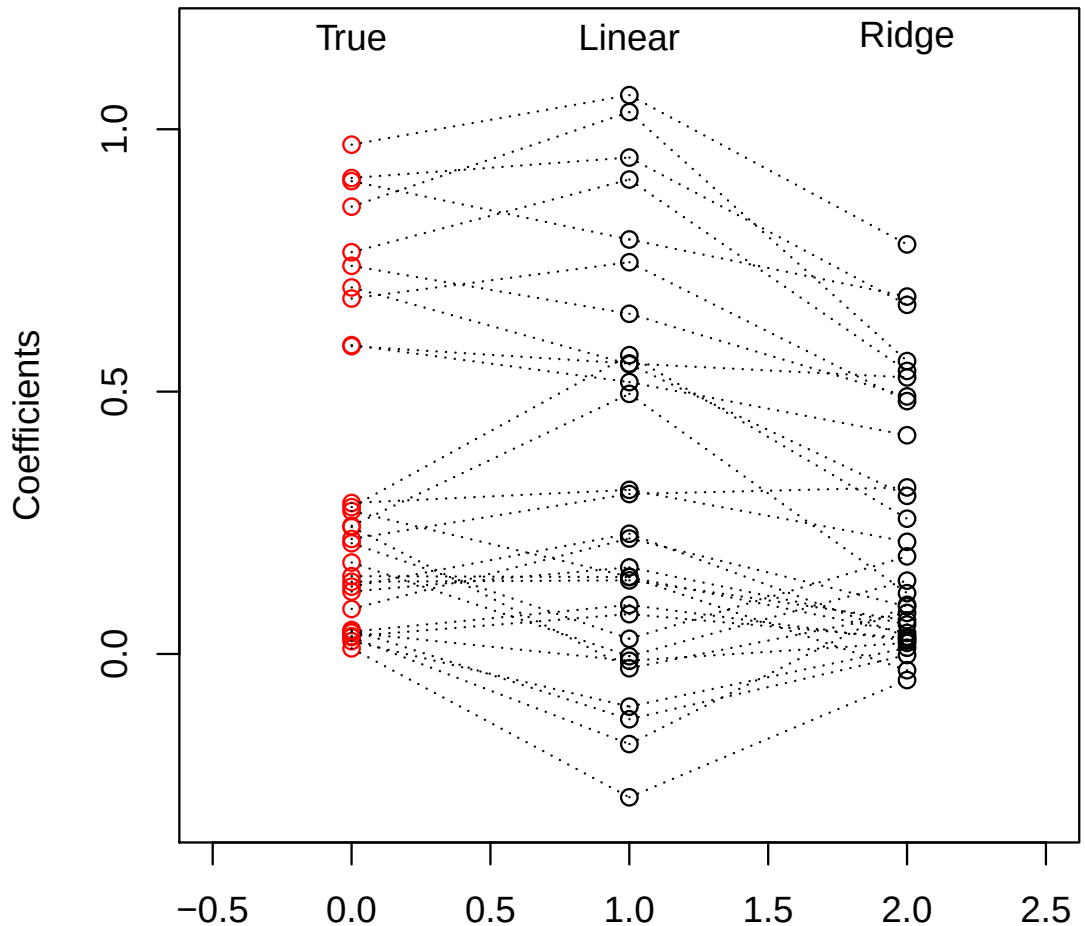
Fixing linear regression in high dimensions

How do we reduce this error?

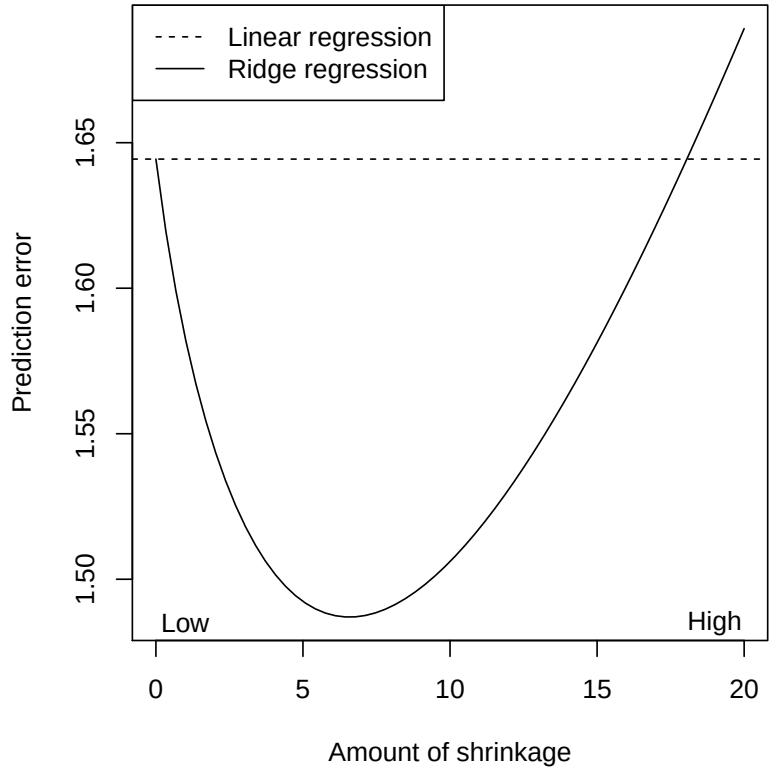
$$\mathbb{E}\left((Y - \hat{\mu}(X))^2\right) = \sigma_x^2 + \text{Bias}\left(\hat{\mu}(X)\right)^2 + \text{Var}\left(\hat{\mu}(X)\right)$$

Example: visual representation of ridge coefficients

A visual representation of the ridge regression coefficients for the same example ($n = 50$, $p = 30$, and $\sigma^2 = 1$; 10 large true coefficients, 20 small) at $\lambda = 25$:



Does it work?



Linear regression:

Squared bias $= 0$

Variance ≈ 0.633

Pred. error $\approx 1 + 0.633$

≈ 1.633

Ridge regression, at its best:

Squared bias ≈ 0.077

Variance ≈ 0.403

Pred. error $\approx 1 + 0.077 + 0.403$

≈ 1.48

This is upsetting! And amazing!

Ridge regression

Ridge regression is like least squares but shrinks the estimated coefficients towards zero. Given a response vector $y \in \mathbb{R}^n$ and a predictor matrix $X \in \mathbb{R}^{n \times p}$, the ridge regression coefficients are defined as

$$\begin{aligned}\hat{\beta}^{\text{ridge}} &= \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \sum_{i=1}^{n} (y_i - x_i^T \beta)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \\ &= \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \underbrace{\|y - X\beta\|_2^2}_{\text{Loss}} + \lambda \underbrace{\|\beta\|_2^2}_{\text{Penalty}}\end{aligned}$$

Ridge regression

$$\hat{\beta}^{\text{ridge}} = \underset{\beta \in \mathbb{R}^p}{\text{argmin}} \sum_{i=1}^{n} (y_i - x_i^T \beta)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

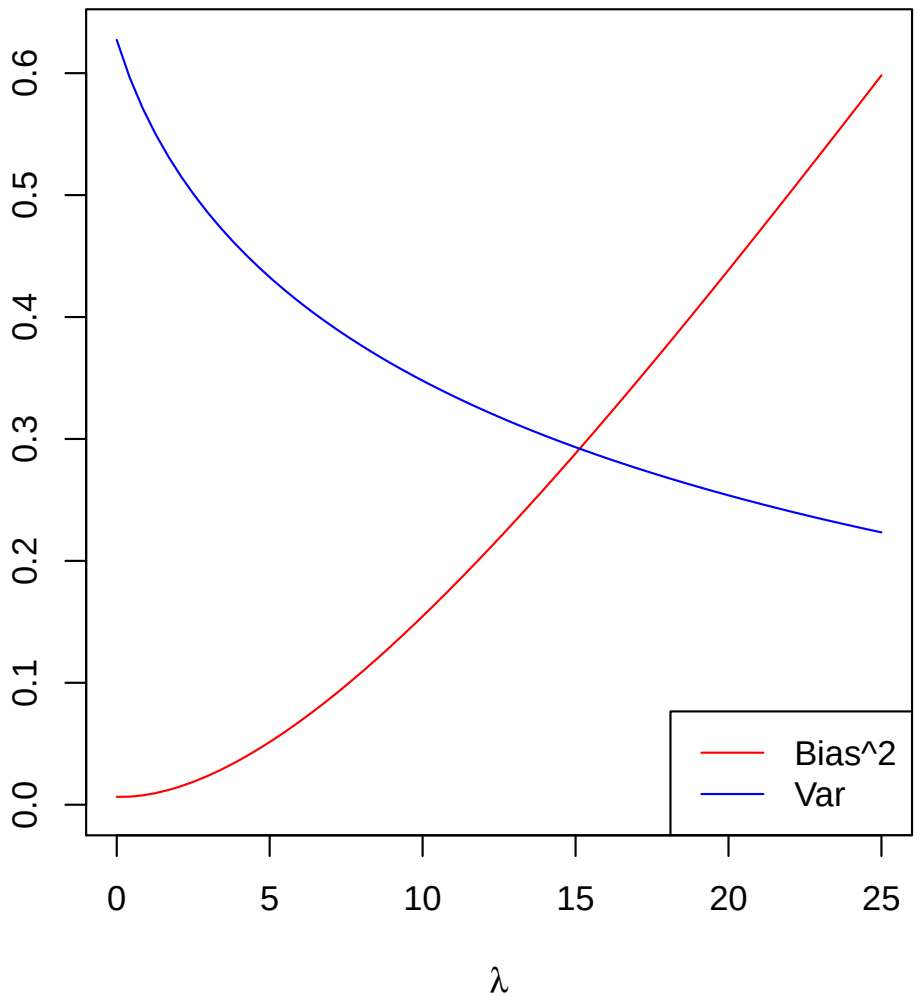
$$= \underset{\beta \in \mathbb{R}^p}{\text{argmin}} \underbrace{\|y - X\beta\|_2^2}_{\text{Loss}} + \lambda \underbrace{\|\beta\|_2^2}_{\text{Penalty}}$$

Here $\lambda \geq 0$ is a tuning parameter, which controls the strength of the penalty term. Note that:

- When $\lambda = 0$,
- When $\lambda = \infty$,
- For λ in between, we are balancing two ideas: fitting a linear model of y on X, and shrinking the coefficients

Example: bias and variance of ridge regression

Bias and variance for our last example ($n = 50$, $p = 30$, $\sigma^2 = 1$; 10 large true coefficients, 20 small):



Important details: centering

When including an intercept term in the regression, we usually leave this coefficient unpenalized.

Otherwise we could add some constant amount c to the vector y, and this would not result in the same solution.

Hence ridge regression with intercept solves

$$\hat{\beta}_0, \hat{\beta}^{\text{ridge}} = \underset{\beta_0 \in \mathbb{R},\, \beta \in \mathbb{R}^p}{\text{argmin}} \; \|y - \beta_0 \mathbf{1} - X\beta\|_2^2 + \lambda \|\beta\|_2^2$$

If we center the columns of X, then the intercept estimate ends up just being $\hat{\beta}_0 = \bar{y}$, so we usually just assume that y, X have been centered and don't include an intercept

Important details: scaling

Also, the penalty term $\|\beta\|_2^2 = \sum_{j=1}^p \beta_j^2$ is unfair is the predictor variables are not on the same scale. (Why?)

Therefore, if we know that the variables are not measured in the same units, we typically scale the columns of X (to have sample variance 1), and then we perform ridge regression

We see that squishing the coefficients toward zero gives:

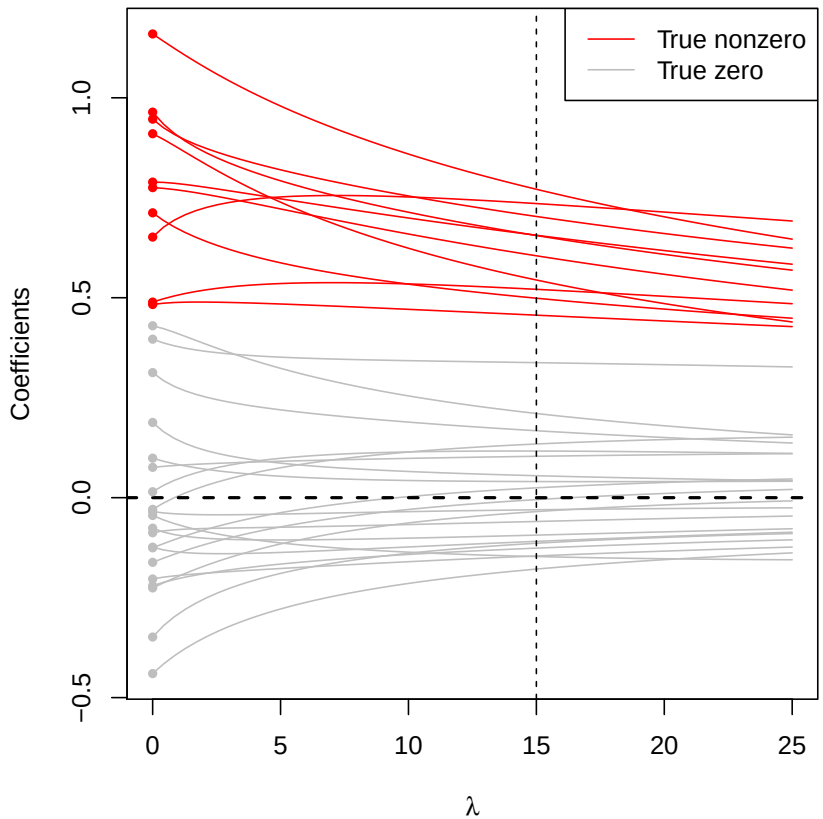
- A decrease in variance!
- A decrease in MSE!
- An increase in bias...

However, ridge regression:

- Seems quite harsh on large, important coefficients.
- Leaves us with a large linear model, which is not that interpretable when we have many variables.

Ridge regression coefficients

Ridge regression coefficients, plotted against λ:



The red paths correspond to the true nonzero coefficients; the gray paths correspond to true zeros. The vertical dashed line at $\lambda = 15$ marks the point above which ridge regression's MSE starts losing to that of linear regression

An important thing to notice is that the gray coefficient paths are not exactly zero; they are shrunken, but still nonzero

The lasso

The lasso[1] estimate is defined as

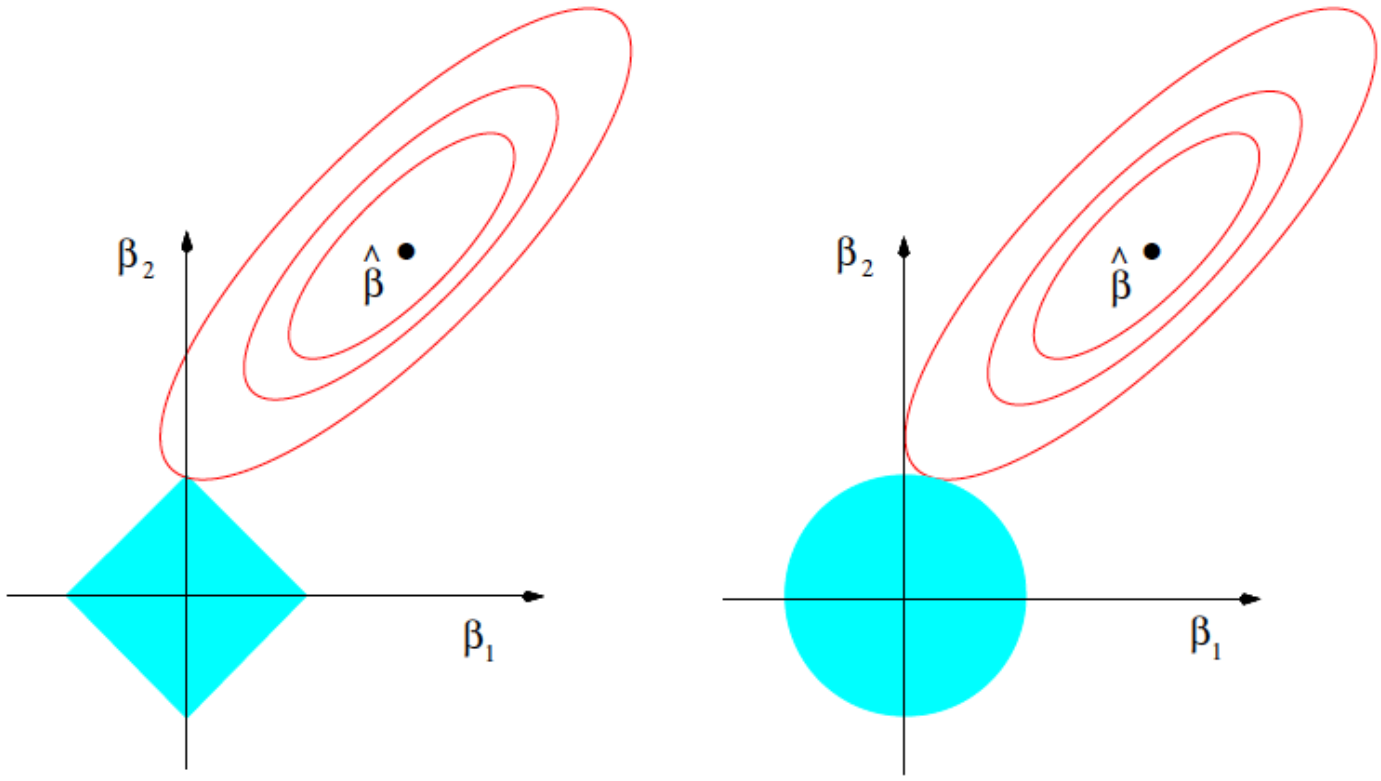
$$\hat{\beta}^{\text{lasso}} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \ \|y - X\beta\|_2^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

$$= \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \ \underbrace{\|y - X\beta|_2^2}_{\text{Loss}} + \lambda \underbrace{\|\beta\|_1}_{\text{Penalty}}$$

The squared ℓ_2 penalty $\|\beta\|_2^2$ of ridge regression, has been replaced by an ℓ_1 penalty $\|\beta\|_1$. Even though these problems look similar, their solutions behave very differently

Note the name "lasso" is actually an acronym for: Least Absolute Selection and Shrinkage Operator

[1] Tibshirani (1996), "Regression Shrinkage and Selection via the Lasso"


β_2
$\hat{\beta}$
β_1
β_2
$\hat{\beta}$
β_1

$$\hat{\beta}^{\text{lasso}} = \underset{\beta \in \mathbb{R}^p}{\text{argmin}} \ \|y - X\beta\|_2^2 + \lambda \|\beta\|_1$$

The tuning parameter λ controls the strength of the penalty.

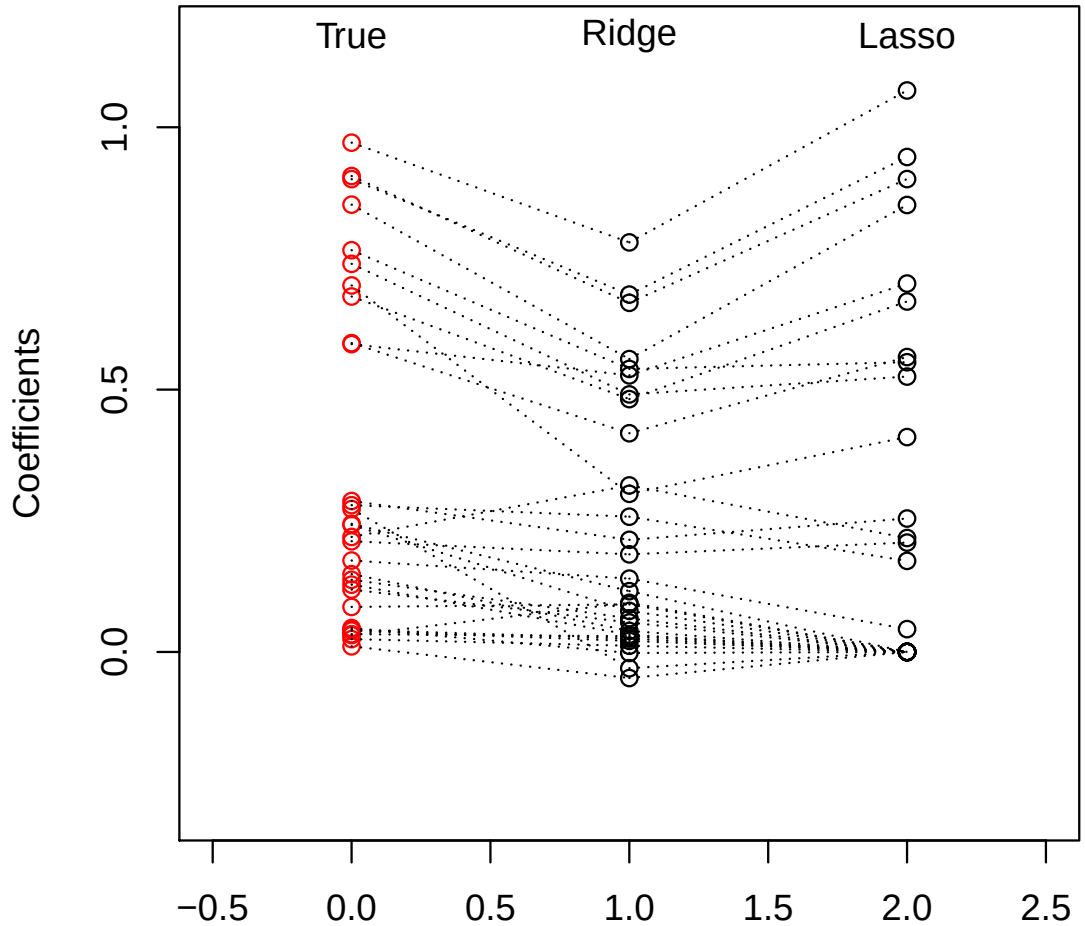
- If $\lambda = 0$:
- As $\lambda \to \infty$:

For λ in between these two extremes, we are balancing two ideas: fitting a linear model of y on X, and shrinking the coefficients. But the nature of the ℓ_1 penalty causes some coefficients to be shrunken to zero exactly

This leads to variable selection!

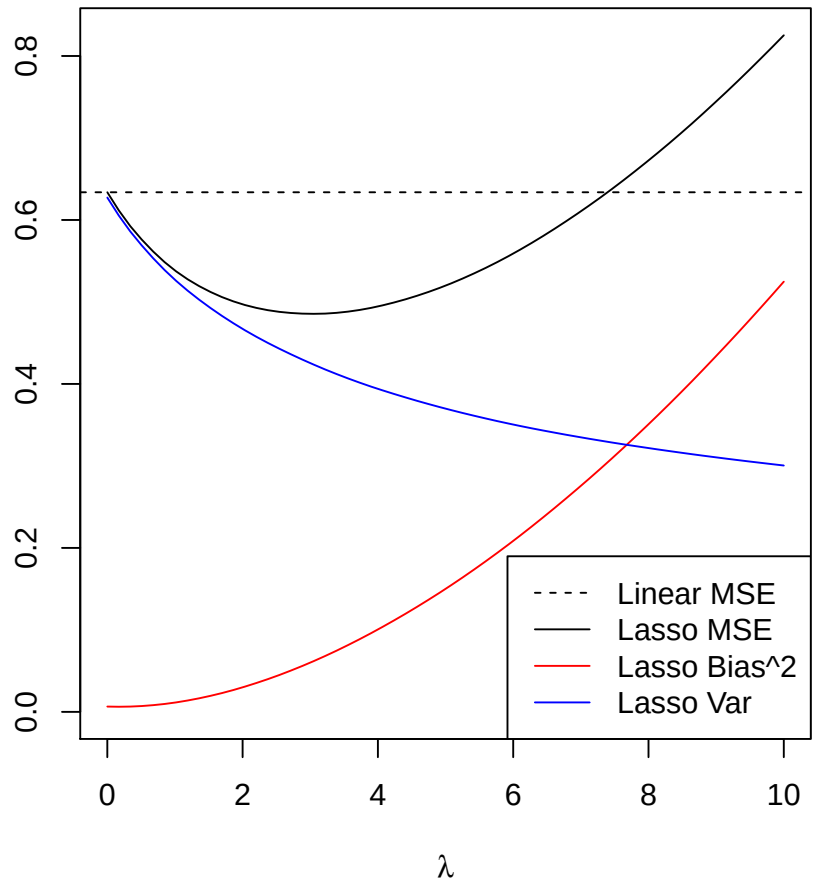
Example: visual representation of lasso coefficients

Our running example from last time: $n = 50$, $p = 30$, $\sigma^2 = 1$, 10 large true coefficients, 20 small. Here is a visual representation of lasso vs. ridge coefficients (with the same degrees of freedom):



Example: subset of small coefficients

Example: $n = 50$, $p = 30$; true coefficients: 10 large, 20 small



Advantages of sparsity

- Interpretability: We can understand what the model relies on for prediction (understanding $\hat{f}$)
- We might gain some insight into the underlying data (though not causally) (helping to understand f)
- If we're building a predictive score, we can measure fewer things in the future (simpler $\hat{f}$ to apply later)